Thread

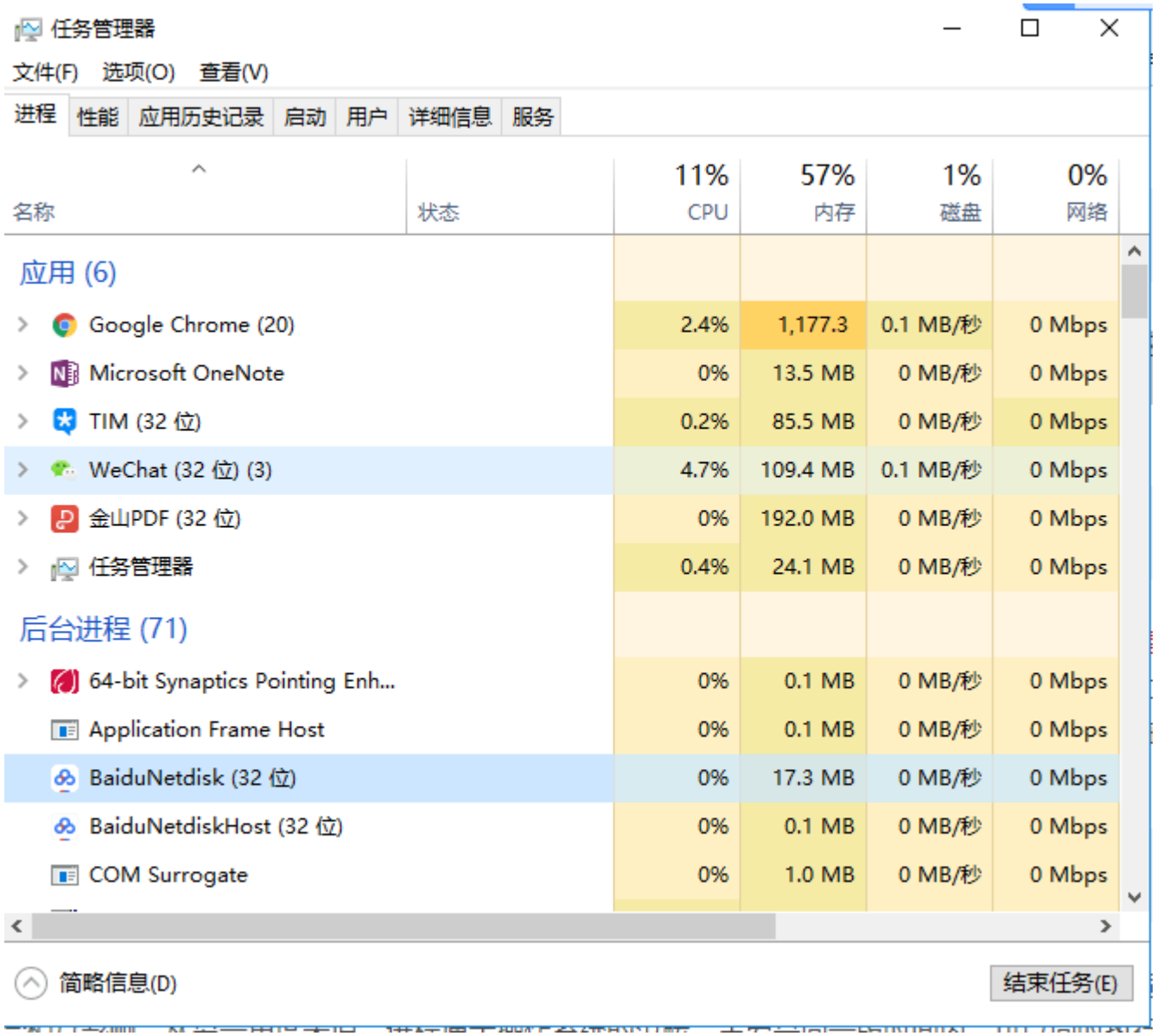
⭐ 什么是 Thread 和 Process？

何为 Process？

Process 是程序的一次执行过程，是系统运行程序的基本单位，因此 Process 是动态的。系统运行一个程序即是一个 Process 从创建，运行到消亡的过程。

在 Java 中，当我们启动 main Function 时其实就是启动了一个 Java Virtual Machine (JVM) 的 Process，而 main Function 所在的 Thread 就是这个 Process 中的一个 Thread，也称 Main Thread。

如下图所示，在 Windows 中通过查看 Task 管理器的方式，我们就可以清楚看到 Windows 当前运行的 Process（`.exe` 文件的运行）。



何为 Thread？

Thread 与 Process 相似，但 Thread 是一个比 Process 更小的执行单位。一个 Process 在其执行的过程中可以产生多个 Thread。与 Process 不同的是同类的多个 Thread Shared Process 的**Heap**和**Method Area**资

源，但每个 Thread 有自己的**Program Counter**、**JVM Stack**和**Native Method Stack**，所以系统在产生一个 Thread，或是在各个 Thread 之间做切换工作时，负担要比 Process 小得多，也正因为如此，Thread 也被称为轻量级 Process。

Java 程序天生就是 Multithreading 程序，我们可以通过 JMX 来看看一个普通的 Java 程序有哪些 Thread，代码如下。

```
public class MultiThread {
    public static void main(String[] args) {
        // 获取 Java 线程管理 MXBean
    ThreadMXBean threadMXBean = ManagementFactory.getThreadMXBean();
        // 不需要获取同步的 monitor 和 synchronizer 信息，仅获取线程和线程堆栈信息
        ThreadInfo[] threadInfos = threadMXBean.dumpAllThreads(false,
false);
        // 遍历线程信息，仅打印线程 ID 和线程名称信息
        for (ThreadInfo threadInfo : threadInfos) {
            System.out.println("[" + threadInfo.getThreadId() + "] " +
threadInfo.getThreadName());
        }
    }
}
```

上述程序输出如下（输出内容可能不同，不用太纠结下面每个 Thread 的作用，只用知道 main Thread 执行 main Method 即可）：

```
[5] Attach Listener //添加事件
[4] Signal Dispatcher // 分发处理给 JVM 信号的线程
[3] Finalizer //调用对象 finalize 方法的线程
[2] Reference Handler //清除 reference 线程
[1] main //main 线程,程序入口
```

从上面的输出内容可以看出：**一个 Java 程序的运行是 main Thread 和多个其他 Thread 同时运行**。

Java Thread 和 Operating System 的 Thread 有啥区别？

早期 JDK 曾使用绿色 Thread（Green Threads）实现用户级 Thread。后来，HotSpot 中通过 `new Thread()` 创建的传统 Thread 采用 Platform Thread 实现，Platform Thread 通常以 1:1 方式映射到 Operating System Thread，由 Operating System 负责调度。Java 21 正式引入的 Virtual Thread 则由 JVM 调度，大量 Virtual Thread 可以复用较少的 Platform Thread 作为载体，因此不能再把所有 Java Thread 都等同于 Operating System Thread。

我们上面提到了 User Thread 和内核 Thread，考虑到很多读者不太了解二者的区别，这里简单介绍一下：

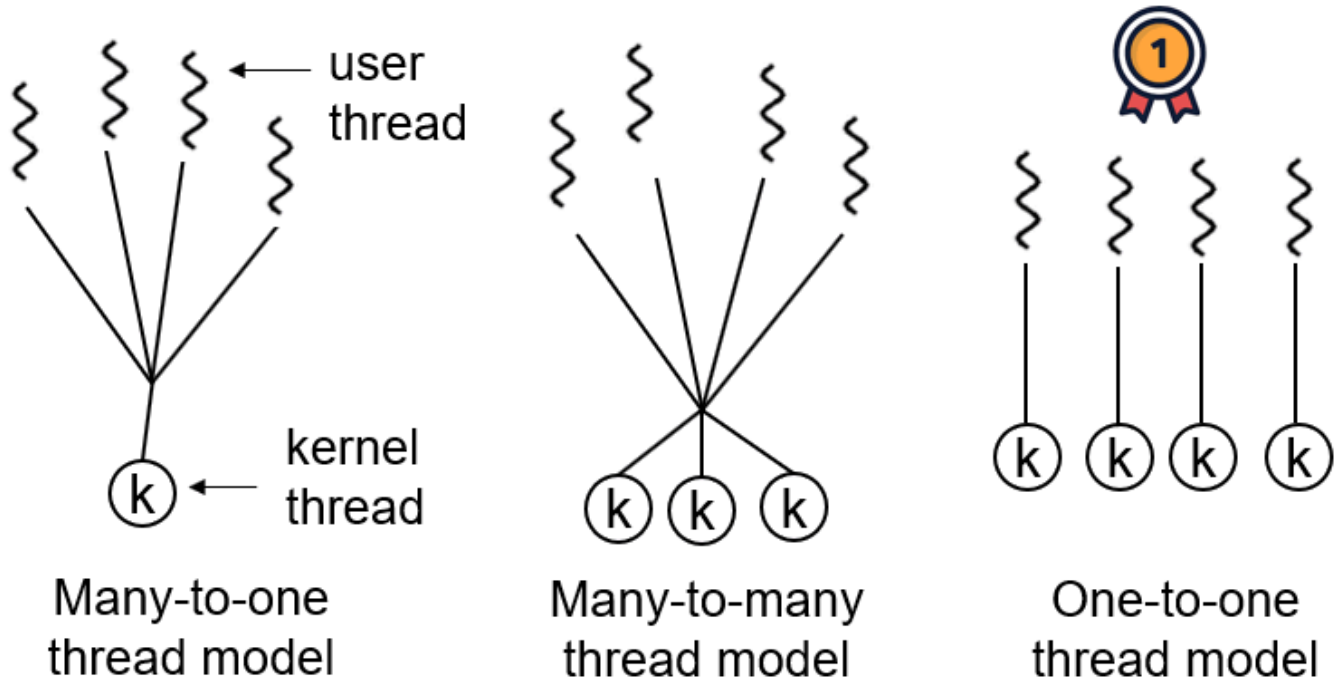
- User Thread：由用户空间程序管理和调度的 Thread，运行在用户空间（专门给应用程序使用）。
- 内核 Thread：由 Operating System 内核管理和调度的 Thread，运行在内核空间（只有内核程序可以访问）。

顺便简单总结一下 User Thread 和内核 Thread 的区别和特点：用户级 Thread 通常由运行时在用户空间调度，创建和切换成本较低；能否利用多核取决于 User Thread 与内核 Thread 之间的映射模型，多对一模型不能 Parallelism 利用多核，多对多模型则可以。内核 Thread 由 Operating System 调度，创建和切换成本通常更高，可以直接利用多核。

一句话概括 Java Thread 和 Operating System Thread 的关系：**Platform Thread 通常映射到 Operating System Thread，而 Virtual Thread 由 JVM 调度并挂载到 Platform Thread 上执行。**

Thread 模型是 User Thread 和内核 Thread 之间的关联方式，常见的 Thread 模型有这三种：

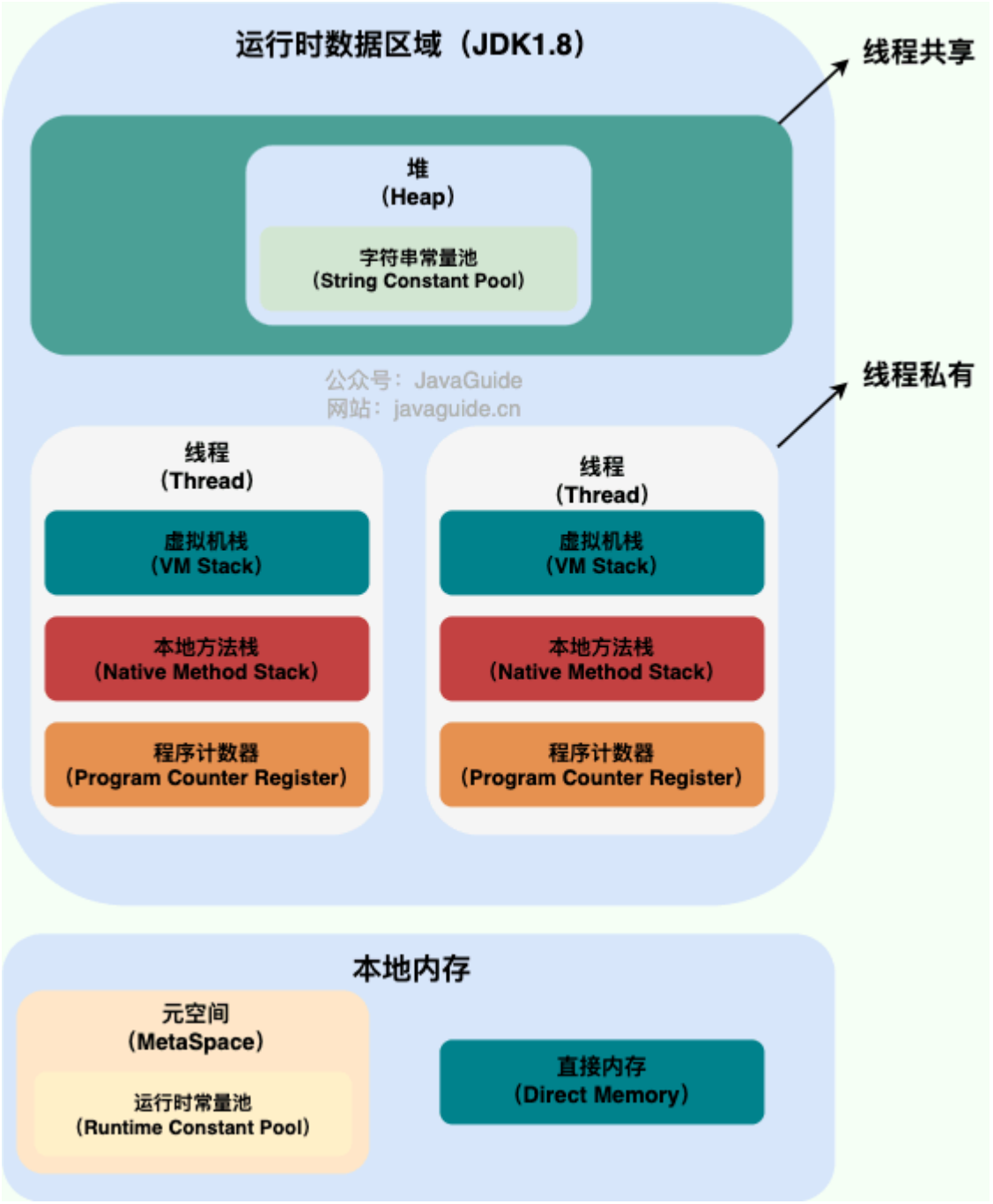
1. 一对一（一个 User Thread 对应一个内核 Thread）
2. 多对一（多个 User Thread 映射到一个内核 Thread）
3. 多对多（多个 User Thread 映射到多个内核 Thread）



在 Windows 和 Linux 等主流 Operating System 中，HotSpot 的 Platform Thread 通常采用一对一模型，也就是一个 Platform Thread 对应一个 Operating System Thread。 Virtual Thread 不采用这种一对一映射，而是由 JVM 调度到一组 Platform Thread 上执行。

⭐ 请简要描述 Thread 与 Process 的关系，区别及优缺点？

下图是 Java Memory Area，通过下图我们从 JVM 的角度来说一下 Thread 和 Process 之间的关系。



从上图可以看出：一个 Process 中可以有多个 Thread，多个 Thread Shared Process 的**Heap**和**Method Area（JDK1.8 之后的 Metaspace）资源，但是每个 Thread 有自己的Program Counter**、**JVM Stack** 和 **Native Method Stack**。

总结： Thread 是 Process 划分成的更小的运行单位。 Thread 和 Process 最大的不同在于基本上各 Process 是独立的，而各 Thread 则不一定，因为同一 Process 中的 Thread 极有可能会相互影响。 Thread 执行 Overhead 小，但不利于资源的管理和保护；而 Process 正相反。

下面是该知识点的扩展内容！

下面来思考这样一个问题：为什么**Program Counter**、**JVM Stack**和**Native Method Stack**是 Thread 私有的呢？为什么 Heap 和 Method Area 是 Thread Shared 的呢？

Program Counter 为什么是私有的？

Program Counter 主要有下面两个作用：

1. Bytecode Interpreter 通过改变 Program Counter 来依次读取 Instruction，从而实现代码的流程控制，如：顺序执行、选择、Loop、Exception Handling。
2. 在 Multithreading 的情况下，Program Counter 用于记录 Current Thread 执行的位置，从而当 Thread 被切换回来的时候能够知道该 Thread 上次运行到哪儿了。

需要注意的是，如果执行的是 native Method，那么 Program Counter 记录的是 undefined Address，只有执行的是 Java 代码时 Program Counter 记录的才是下一条 Instruction 的 Address。

所以，Program Counter 私有主要是为了**Thread 切换后能恢复到正确的执行位置**。

JVM Stack 和 Native Method Stack 为什么是私有的？

- **JVM Stack：** 每个 Java Method 在执行之前会创建一个 Stack Frame 用于存储 Local Variable Table、Operand Stack、Constant Pool Reference 等信息。从 Method Invocation 直至执行完成的过程，就对应着一个 Stack Frame 在 JVM Stack 中 Push 和 Pop 的过程。
- **Native Method Stack：** 和 JVM Stack 所发挥的作用非常相似，区别是：**JVM Stack 为虚拟机执行 Java Method（也就是 Bytecode）服务，而 Native Method Stack 则为虚拟机使用到的 Native Method 服务。** 在 HotSpot 虚拟机中和 JVM Stack 合二为一。

所以，为了**保证 Thread 中的 Local Variable 不被别的 Thread 访问到**，JVM Stack 和 Native Method Stack 是 Thread 私有的。

一句话简单了解 Heap 和 Method Area

Heap 和 Method Area 是所有 Thread Shared 的资源，其中 Heap 是 Process 中最大的一块内存，主要用于存放新创建的 Object（几乎所有 Object 都在这里分配内存），Method Area 主要用于存放已被加载的 Class 信息、Constant、Static Variable、Just-In-Time Compiler 编译后的代码等数据。

如何创建 Thread？

一般来说，创建 Thread 有很多种方式，例如 Inheritance `Thread` Class、实现 `Runnable` Interface、实现 `Callable` Interface、使用 Thread Pool、使用 `CompletableFuture` Class 等等。

不过，这些方式其实并没有真正创建出 Thread。准确点来说，这些都属于是在 Java 代码中使用多线程的方法。

严格来说，Java 就只有一种方式可以创建 Thread，那就是通过 `new Thread().start()` 创建。不管是哪种方式，最终还是依赖于 `new Thread().start()`。

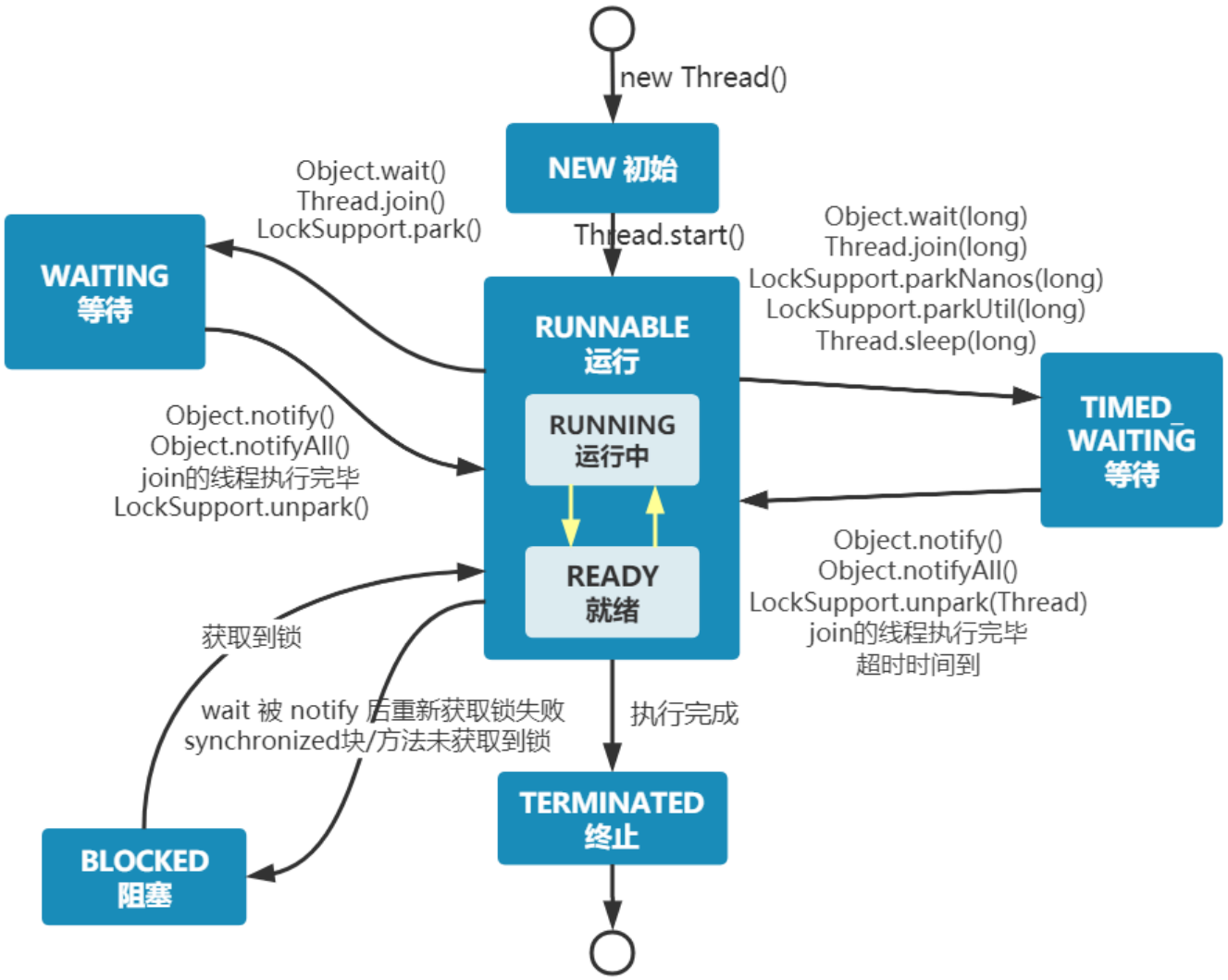
⭐ 说说 Thread 的 Lifecycle 和状态？

Java Thread 在运行的 Lifecycle 中的指定时刻只可能处于下面 6 种不同状态的其中一个状态：

- NEW: 初始状态，Thread 被创建出来但没有被调用 `start()`。
- RUNNABLE: 运行状态，Thread 被调用了 `start()` 等待运行的状态。
- BLOCKED：Blocking 状态，需要等待 Lock 释放。
- WAITING：等待状态，表示该 Thread 需要等待其他 Thread 做出一些特定动作（通知或 Interrupt）。
- TIMED_WAITING：Timeout 等待状态，可以在指定的时间后自行返回而不是像 WAITING 那样一直等待。
- TERMINATED：终止状态，表示该 Thread 已经运行完毕。

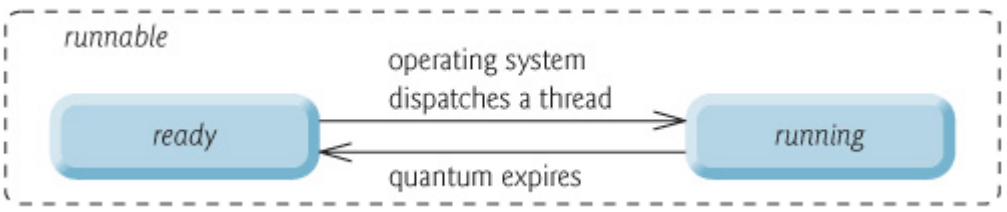
Thread 在 Lifecycle 中并不是固定处于某一个状态而是随着代码的执行在不同状态之间切换。

Java Thread State 变迁图(图源：挑错 |《Java Concurrent Programming 的艺术》中关于 Thread State 的三处错误)：



由上图可以看出：Thread 创建之后它将处于 **NEW（新建）** 状态，调用 `start()` Method 后开始运行，Thread 这时候处于 **READY（可运行）** 状态。可运行状态的 Thread 获得了 CPU Time Slice（timeslice）后就处于 **RUNNING（运行）** 状态。

> 在 Operating System 层面，Thread 有 READY 和 RUNNING 状态；而在 JVM 层面，只能看到 RUNNABLE 状态（图源：HowToDoInJava：Java Thread Life Cycle and Thread States），所以 Java 系统一般将这两个状态统称为 **RUNNABLE（运行中）** 状态。
>
> **为什么 JVM 没有区分这两种状态呢？** Java 的 `Thread.State` 描述的是 JVM 层面的 Thread State，不用于反映 Operating System Scheduler 的全部内部状态。具体调度策略、Time Slice Length 和是否采用轮转方式都由 Operating System 及其配置决定，不能固定概括为 10～20 ms 的轮转调度。



- 当 Thread 执行 `wait()` Method 之后，Thread 进入 **WAITING（等待）** 状态。进入等待状态的 Thread 需要依靠其他 Thread 的通知才能够返回到运行状态。
- **TIMED_WAITING（Timeout 等待）** 状态相当于在等待状态的基础上增加了 Timeout 限制，比如通过 `sleep (long millis)` Method 或 `wait (long millis)` Method 可以将 Thread 置于 TIMED_WAITING 状态。当 Timeout 时间结束后，Thread 将会返回到 RUNNABLE 状态。
- 当 Thread 进入 `synchronized` Method/块或者调用 `wait` 后（被 `notify`）重新进入 `synchronized` Method/块，但是 Lock 被其它 Thread 占有，这个时候 Thread 就会进入 **BLOCKED（Blocking）** 状态。
- Thread 在执行完了 `run()` Method 之后将会进入到 **TERMINATED（终止）** 状态。

什么是 Thread Context Switch？

Thread 在执行过程中会有自己的运行条件和状态（也称 Context），比如上文所说到过的 Program Counter，Stack 信息等。当出现如下情况的时候，Thread 会从占用 CPU 状态中退出。

- 主动让出 CPU，比如调用了 `sleep()`, `wait()` 等。
- Time Slice 用完，因为 Operating System 要防止一个 Thread 或者 Process 长时间占用 CPU 导致其他 Thread 或者 Process 饿死。
- 调用了 Blocking Type 的系统 Interrupt，比如请求 IO，Thread 被 Blocking。
- 被终止或结束运行

这其中前三种都会发生 Thread 切换，Thread 切换意味着需要保存 Current Thread 的 Context，留待 Thread 下次占用 CPU 的时候恢复现场。并加载下一个将要占用 CPU 的 Thread Context。这就是所谓的 **Context Switch**。

Context Switch 是现代 Operating System 的基本功能，因其每次需要保存信息恢复信息，这将会占用 CPU，内存等系统资源进行处理，也就意味着效率会有一定损耗，如果频繁切换就会造成整体效率低下。

Thread#sleep() Method 和 Object#wait() Method 对比

共同点：两者都可以暂停 Thread 的执行。

区别：

- **`sleep()` Method 没有 Release Lock，而 `wait()` Method 释放了 Lock**。
- `wait()` 通常被用于 Thread 间交互/通信，`sleep()` 通常被用于暂停执行。
- `wait()` Method 被调用后，Thread 不会自动苏醒，需要别的 Thread 调用同一个 Object 上的 `notify()` 或者 `notifyAll()` Method。`sleep()` Method 执行完成后，Thread 会自动苏醒，或者也可以使用 `wait(long timeout)` Timeout 后 Thread 会自动苏醒。
- `sleep()` 是 `Thread` Class 的静态 Native Method，`wait()` 则是 `Object` Class 的 Native Method。为什么这样设计呢？下一个问题就会聊到。

为什么 wait() Method 不定义在 Thread 中？

`wait()` 是让获得 Object Lock 的 Thread 实现等待，会自动释放 Current Thread 占有的 Object Lock。每个 Object（`Object`）都拥有 Object Lock，既然要释放 Current Thread 占有的 Object Lock 并让其进入 WAITING 状态，自然是要操作对应的 Object（`Object`）而非当前的 Thread（`Thread`）。

类似的问题：**为什么 `sleep()` Method 定义在 `Thread` 中？**

因为 `sleep()` 是让 Current Thread 暂停执行，不涉及到 Object Class，也不需要获得 Object Lock。

可以直接调用 Thread Class 的 run Method 吗？

这是另一个非常经典的 Java Multithreading 面试问题，而且在面试中会经常被问到。很简单，但是很多人都会答不上来！

new 一个 `Thread`，Thread 进入了新建状态。调用 `start()` Method，会启动一个 Thread 并使 Thread 进入了就绪状态，当分配到 Time Slice 后就可以开始运行了。 `start()` 会执行 Thread 的相应准备工作，然后自动执行 `run()` Method 的内容，这是真正的 Multithreading 工作。 但是，直接执行 `run()` Method，会把 `run()` Method 当成一个普通 Method 在调用该 Method 的 Thread 去执行，所以这并不是 Multithreading 工作。

总结：调用 `start()` Method 方可启动 Thread 并使 Thread 进入就绪状态，直接执行 `run()` Method 的话不会以 Multithreading 的方式执行。

Multithreading

Concurrency 与 Parallelism 的区别

- **Concurrency**：两个及两个以上的作业在同一 **时间段** 内执行。
- **Parallelism**：两个及两个以上的作业在同一 **时刻** 执行。

最关键的点是：是否是 **同时** 执行。

同步和 Asynchronous 的区别

- **同步**：发出一个调用之后，在没有得到结果之前， 该调用就不可以返回，一直等待。
- **Asynchronous**：调用在发出之后，不用等待返回结果，该调用直接返回。

⭐ 为什么要使用 Multithreading？

先从总体上来说：

- **从计算机底层来说：** Thread 可以比作是轻量级的 Process，是程序执行的最小单位，Thread 间的切换和调度的成本远远小于 Process。另外，多核 CPU 时代意味着多个 Thread 可以同时运行，这减少了 Thread Context Switch 的 Overhead。
- **从当代互联网发展趋势来说：** 现在的系统动不动就要求百万级甚至千万级的 Concurrency 量，而 Multithreading Concurrent Programming 正是开发 High Concurrency 系统的基础，利用好 Multithreading 机制可以大大提高系统整体的 Concurrency 能力以及 Performance。

再深入到计算机底层来探讨：

- **单核时代**：在单核时代 Multithreading 主要是为了提高单 Process 利用 CPU 和 IO 系统的效率。 假设只运行了一个 Java Process 的情况，当我们请求 IO 的时候，如果 Java Process 中只有一个 Thread，此 Thread 被 IO Blocking 则整个 Process 被 Blocking。CPU 和 IO 设备只有一个在运行，那么可以简单地说系统整体效率只有 50%。当使用 Multithreading 的时候，一个 Thread 被 IO Blocking，其他 Thread 还可以继续使用 CPU。从而提高了 Java Process 利用系统资源的整体效率。
- **多核时代**: 多核时代 Multithreading 主要是为了提高 Process 利用多核 CPU 的能力。举个例子：假如我们要计算一个复杂的 Task，我们只用一个 Thread 的话，不论系统有几个 CPU 核心，都只会有一个 CPU 核心被利用到。而创建多个 Thread，这些 Thread 可以被映射到底层多个 CPU 核心上执行，在

Task 中的多个 Thread 没有资源竞争的情况下，Task 执行的效率会有显著性的提高，约等于（单核时执行时间/CPU 核心数）。

⭐ 单核 CPU 支持 Java Multithreading 吗？

单核 CPU 是支持 Java Multithreading 的。 Operating System 通过 Round-Robin 的方式，将 CPU 的时间分配给不同的 Thread。尽管单核 CPU 一次只能执行一个 Task，但通过快速在多个 Thread 之间切换，可以让用户感觉多个 Task 是同时进行的。

这里顺带提一下 Java 使用的 Thread Scheduling 方式。

Operating System 主要通过两种 Thread Scheduling 方式来管理 Multithreading 的执行：

- **Preemptive Scheduling**：Operating System 决定何时暂停当前正在运行的 Thread，并切换到另一个 Thread 执行。这种切换通常是由系统时钟 Interrupt（Round-Robin）或其他高优先级事件（如 I/O 操作完成）触发的。这种方式存在 Context Switch Overhead，但 Fairness 和 CPU 资源利用率较好，不易 Blocking。
- **Cooperative Scheduling**：Thread 执行完毕后，主动通知系统切换到另一个 Thread。这种方式可以减少 Context Switch 带来的 Performance Overhead，但 Fairness 较差，容易 Blocking。

Java 使用的 Thread Scheduling 是抢占式的。也就是说，JVM 本身不负责 Thread 的调度，而是将 Thread 的调度委托给 Operating System。 Operating System 通常会基于 Thread 优先级和 Time Slice 来调度 Thread 的执行，高优先级的 Thread 通常获得 CPU Time Slice 的机会更多。

⭐ 单核 CPU 上运行多个 Thread 效率一定会高吗？

单核 CPU 同时运行多个 Thread 的效率是否会高，取决于 Thread 的 Type 和 Task 的性质。一般来说，有两种 Type 的 Thread：

1. **CPU 密集型**：CPU 密集型的 Thread 主要进行计算和逻辑处理，需要占用大量的 CPU 资源。
2. **IO 密集型**：IO 密集型的 Thread 主要进行输入输出操作，如读写文件、网络通信等，需要等待 IO 设备的响应，而不占用太多的 CPU 资源。

在单核 CPU 上，同一时刻只能有一个 Thread 在运行，其他 Thread 需要等待 CPU 的 Time Slice 分配。如果 Thread 是 CPU 密集型的，那么多个 Thread 同时运行会导致频繁的 Thread 切换，增加了系统的 Overhead，降低了效率。如果 Thread 是 IO 密集型的，那么多个 Thread 同时运行可以利用 CPU 在等待 IO 时的空闲时间，提高了效率。

因此，对于单核 CPU 来说，如果 Task 是 CPU 密集型的，那么开很 Multithreading 会影响效率；如果 Task 是 IO 密集型的，那么开很 Multithreading 会提高效率。当然，这里的"很多"也要适度，不能超过系统能够承受的上限。

使用 Multithreading 可能带来什么问题？

Concurrent Programming 的目的就是为了能提高程序的执行效率进而提高程序的运行速度，但是 Concurrent Programming 并不总是能提高程序运行速度的，而且 Concurrent Programming 可能会遇到很多问题，比如：Memory Leak、Deadlock、Not Thread-safe 等等。

如何理解 Thread-safe 和不安全？

Thread-safe 和不安全是在 Multithreading 环境下对于同一份数据的访问是否能够保证其正确性和一致性的描述。

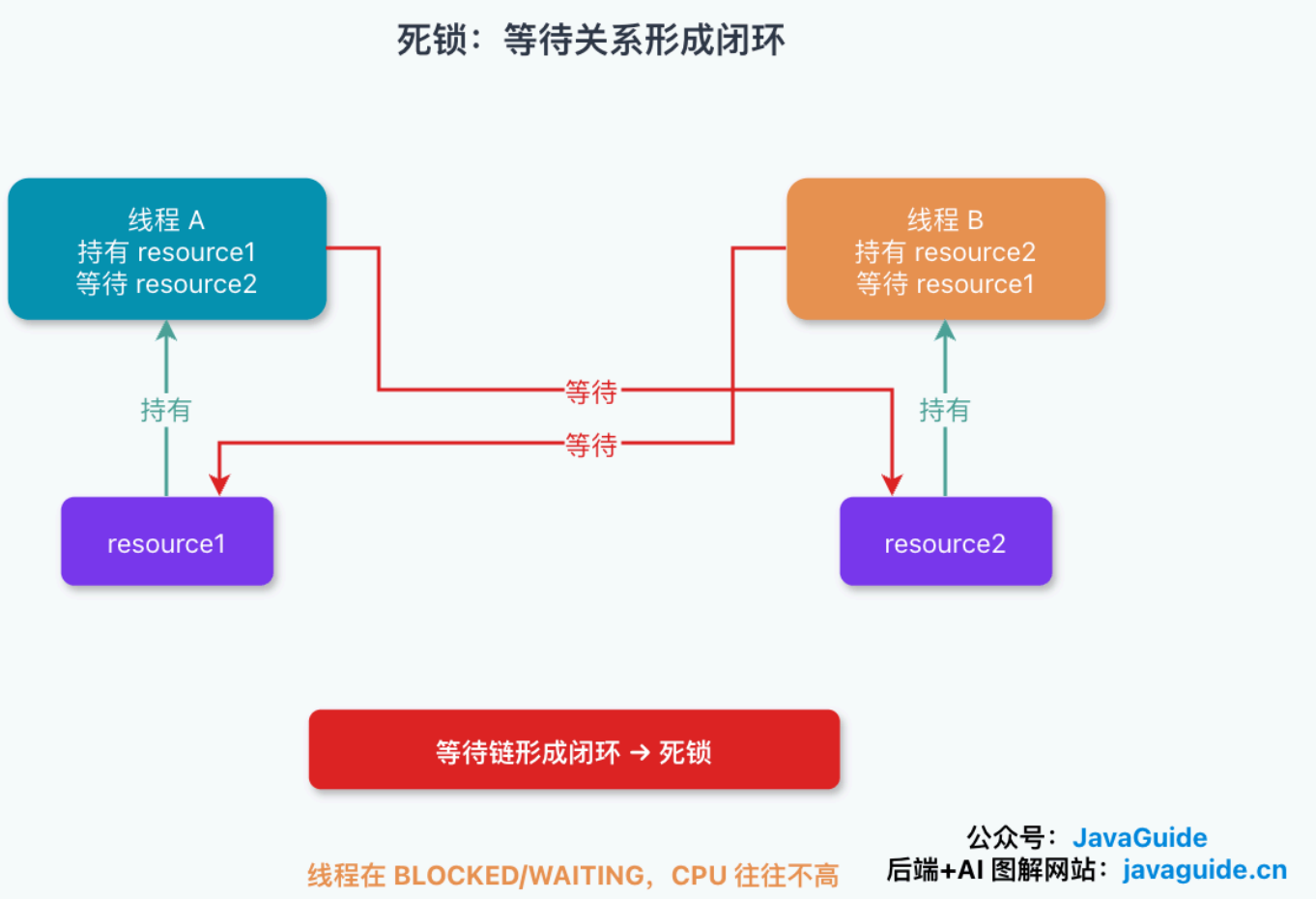
- Thread-safe 指的是在 Multithreading 环境下，对于同一份数据，不管有多少个 Thread 同时访问，都能保证这份数据的正确性和一致性。
- Not Thread-safe 则表示在 Multithreading 环境下，对于同一份数据，多个 Thread 同时访问时可能会导致数据混乱、错误或者丢失。

⭐ Deadlock

什么是 Thread Deadlock？

Thread Deadlock 描述的是这样一种情况：多个 Thread 同时被 Blocking，它们中的一个或者全部都在等待某个资源被释放。由于 Thread 被无限期地 Blocking，因此程序不可能正常终止。

如下图所示，Thread A 持有资源 2，Thread B 持有资源 1，他们同时都想申请对方的资源，所以这两个 Thread 就会互相等待而进入 Deadlock 状态。



下面通过一个例子来说明 Thread Deadlock，代码模拟了上图的 Deadlock 的情况（代码来源于《Concurrent Programming 之美》）：

```
public class DeadLockDemo {
    private static Object resource1 = new Object();//资源 1
    private static Object resource2 = new Object();//资源 2

    public static void main(String[] args) {
```

```
        new Thread(() -> {
            synchronized (resource1) {
                System.out.println(Thread.currentThread() + "get
resource1");
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                System.out.println(Thread.currentThread() + "waiting get
resource2");
                synchronized (resource2) {
                    System.out.println(Thread.currentThread() + "get
resource2");
                }
            }
        }, "线程 1").start();

        new Thread(() -> {
            synchronized (resource2) {
                System.out.println(Thread.currentThread() + "get
resource2");
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                System.out.println(Thread.currentThread() + "waiting get
resource1");
                synchronized (resource1) {
                    System.out.println(Thread.currentThread() + "get
resource1");
                }
            }
        }, "线程 2").start();
    }
}
```

Output

```
Thread[线程 1,5,main]get resource1
Thread[线程 2,5,main]get resource2
Thread[线程 1,5,main]waiting get resource2
Thread[线程 2,5,main]waiting get resource1
```

Thread A 通过 `synchronized (resource1)` 获得 `resource1` 的 Monitor Lock，然后通过 `Thread.sleep(1000);` 让 Thread A 休眠 1s，为的是让 Thread B 得到执行然后获取到 resource2 的 Monitor Lock。 Thread A 和 Thread B 休眠结束了都开始企图请求获取对方的资源，然后这两个 Thread 就会陷入互相等待的状态，这也就产生了 Deadlock。

上面的例子符合产生 Deadlock 的四个必要条件：

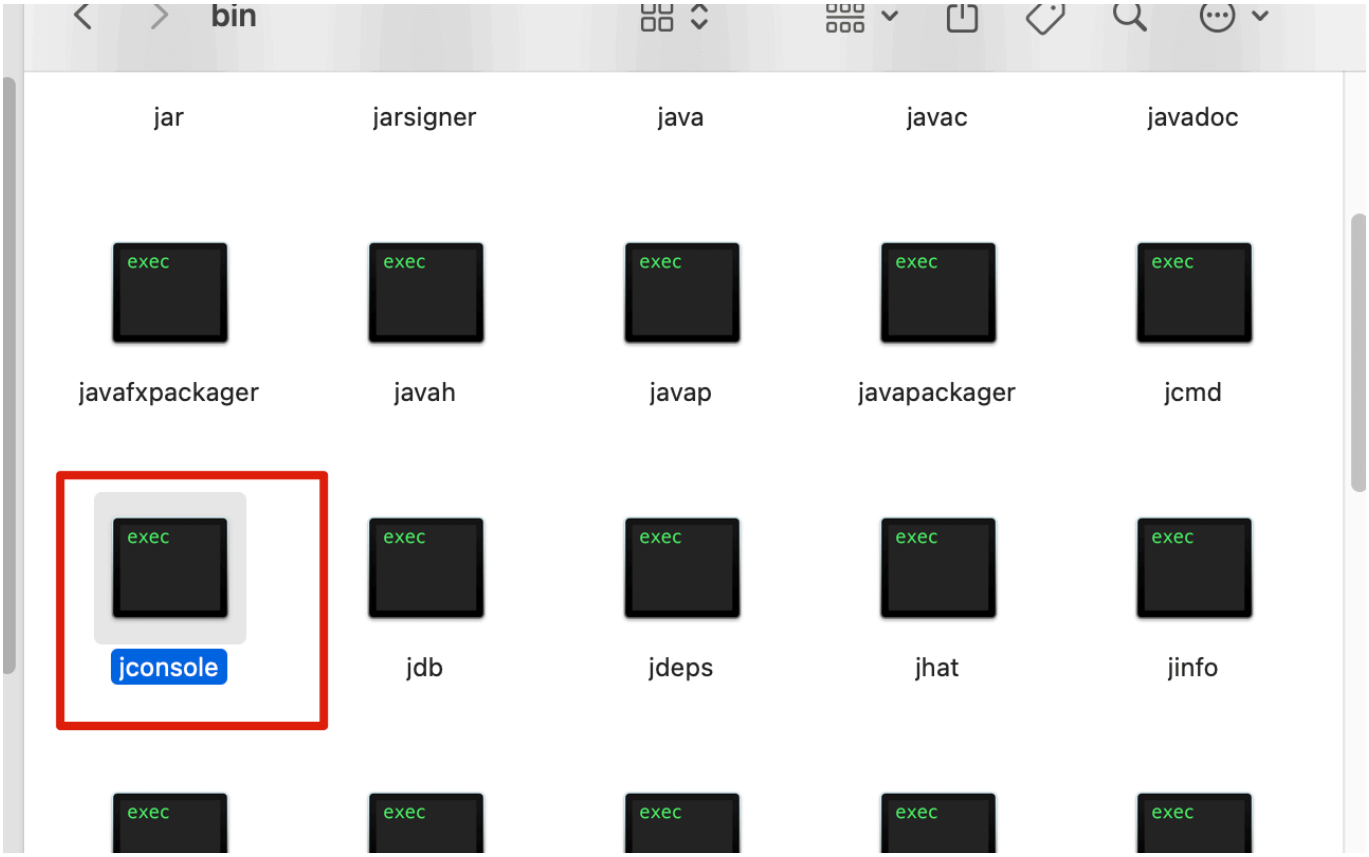
1. **Mutual Exclusion 条件**：该资源任意一个时刻只由一个 Thread 占用。
2. **请求与保持条件**：一个 Thread 因请求资源而 Blocking 时，对已获得的资源保持不放。
3. **不剥夺条件**：Thread 已获得的资源在未使用完之前不能被其他 Thread 强行剥夺，只有自己使用完毕后才释放资源。
4. **Loop 等待条件**：若干 Thread 之间形成一种头尾相接的 Loop 等待资源关系。

如何检测 Deadlock？

- 使用 `jstack <pid>` 或 `jcmd <pid> Thread.print -l` 查看 Thread Stack 和 Concurrency Lock 信息。如果检测到 Java 级别的 Deadlock，输出中会列出相关 Thread 及其持有、等待的 Lock。`jmap` 主要用于查看 Heap 信息或生成 Heap Dump，不是 Thread Deadlock 诊断工具。
- 采用 VisualVM、JConsole 等工具进行排查。

这里以 JConsole 工具为例进行演示。

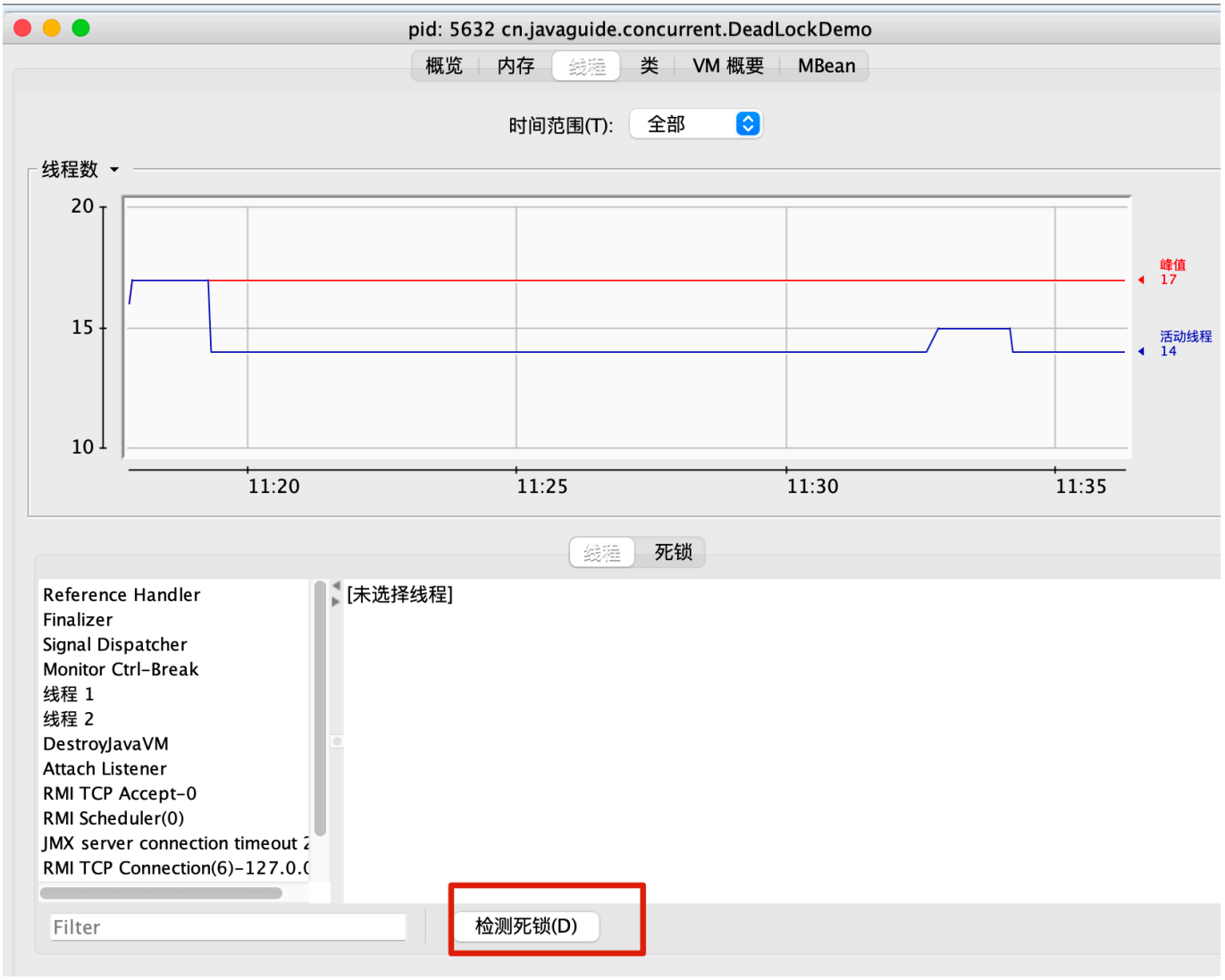
首先，我们要找到 JDK 的 bin 目录，找到 jconsole 并双击打开。

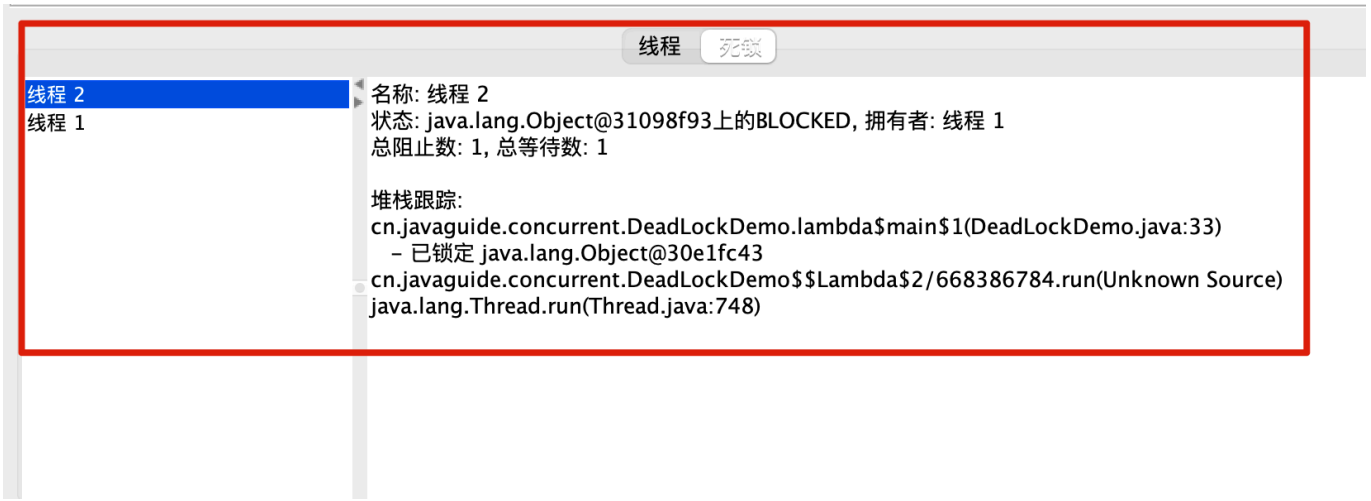


对于 MAC 用户来说，可以通过 `/usr/libexec/java_home -V` 查看 JDK 安装目录，找到后通过 `open . + 文件夹地址` 打开即可。例如，我本地的某个 JDK 的路径是：

```
 open . /Users/guide/Library/Java/JavaVirtualMachines/corretto-1.8.0_252/Contents/Home
```

打开 jconsole 后，连接对应的程序，然后进入 Thread 界面选择检测 Deadlock 即可！





如何预防和避免 Thread Deadlock？

如何预防 Deadlock？ 破坏 Deadlock 的产生的必要条件即可：

1. **破坏请求与保持条件**：一次性申请所有的资源。
2. **破坏不剥夺条件**：占用部分资源的 Thread 进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
3. **破坏 Loop 等待条件**：靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏 Loop 等待条件。

如何避免 Deadlock？

避免 Deadlock 就是在资源分配时，借助于 Algorithm（比如银行家 Algorithm）对资源分配进行计算评估，使其进入安全状态。

> **安全状态** 指的是系统能够按照某种 Thread 推进顺序（P1、P2、P3.....Pn）来为每个 Thread 分配所需资源，直到满足每个 Thread 对资源的最大需求，使每个 Thread 都可顺利完成。称 <P1、P2、P3.....Pn> 序列为安全序列。

我们对 Thread 2 的代码修改成下面这样就不会产生 Deadlock 了。

```
new Thread(() -> {
            synchronized (resource1) {
                System.out.println(Thread.currentThread() + "get
resource1");
                try {
                    Thread.sleep(1000);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                System.out.println(Thread.currentThread() + "waiting get
resource2");
                synchronized (resource2) {
                    System.out.println(Thread.currentThread() + "get
resource2");
                }
            }
        }, "线程 2").start();
```

输出：

```
Thread[线程 1,5,main]get resource1
Thread[线程 1,5,main]waiting get resource2
Thread[线程 1,5,main]get resource2
Thread[线程 2,5,main]get resource1
Thread[线程 2,5,main]waiting get resource2
Thread[线程 2,5,main]get resource2

Process finished with exit code 0
```

我们分析一下上面的代码为什么避免了 Deadlock 的发生？

Thread 1 首先获得到 resource1 的 Monitor Lock，这时候 Thread 2 就获取不到了。然后 Thread 1 再去获取 resource2 的 Monitor Lock，可以获取到。然后 Thread 1 释放了对 resource1、resource2 的 Monitor Lock 的占用，Thread 2 获取到就可以执行了。这样就破坏了 Loop 等待条件，因此避免了 Deadlock。